Lesson 10 Teacher's Key: Play It Again

Unreal Engine (optional advanced track) - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Extending WBP_HUD (built in Lesson 9) with a third, general-purpose status text element, plus a Custom Event on BP_GameManager (ResetGame) that fully replaces the old inline BeginPlay spawn logic - continuing this course's established push-based widget pattern into genuinely new territory (destroying/respawning actors) this week.

Level 1 - Full Worked Solution

- SetStatus() on WBP_HUD is a small, single-purpose Custom Event - same shape as UpdateHUD() from Lesson 9, just for a different piece of persistent text.
- ResetGame is BP_GameManager's own reusable routine (it owns Score, Lives, EnemiesRemaining and GameActive) - it fully replaces what BeginPlay used to do inline (spawn enemies), and is now called from two places: BeginPlay (via a wrapping call) and the new IA_Restart Enhanced Input Action Event, whose R mapping was simply added to the existing IMC_Default from Lesson 2 - no new mapping-context wiring needed.
- Get All Actors of Class(BP_Enemy) -> Destroy Actor is the standard Blueprint way to find and clear every instance of an Actor class - Unreal's equivalent of Godot's Group query or Unity's tag search.

Highlighted lines are new this week; everything else is code you already have.

```
WBP_HUD's Event Graph (new):
Custom Event SetStatus(StatusText: String) ->
  Set Text(StatusTextBlock, StatusText)

BP_Enemy's Event Graph (revised):
Event Tick ->
  Branch(Get GameActive from Get Actor of Class(BP_GameManager)) -> True:
    Add Actor World Offset(Delta = (0, -EnemySpeed * Delta Seconds, 0))
    Branch(GetActorLocation().Y < -400) -> True: Call HandleEnemyEscaped
on BP_GameManager -> Destroy Actor(Self)

BP_GameManager (variables: Score, Lives, EnemiesRemaining, HUDWidget,
GameActive):
Custom Event ResetGame ->
  For Each (Get All Actors of Class(BP_Enemy)) -> Destroy Actor
  Score = 0
  Lives = 3
  EnemiesRemaining = 5
```

```
  GameActive = true
  Call SetStatus(HUDWidget, "")
  Call UpdateHUD(HUDWidget, Score, Lives)
  For Loop (First Index 0, Last Index 4) -> Spawn Actor from
Class(BP_Enemy) at (BaseX + Index * 150, BaseY, 0) -> Bind Event to
OnEnemyDestroyed / OnEnemyEscaped

Event BeginPlay ->
  Create Widget(WBP_HUD) -> Add to Viewport -> Set HUDWidget
  Call ResetGame

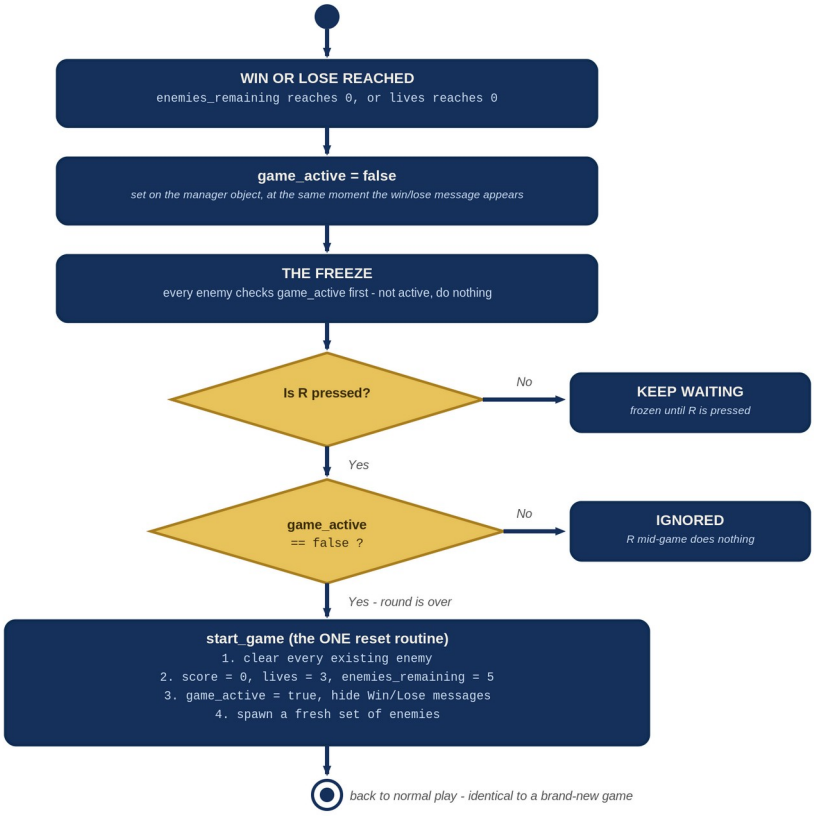
Custom Event HandleEnemyDestroyed ->
  EnemiesRemaining = EnemiesRemaining - 1
  Score = Score + 10
  Call UpdateHUD(HUDWidget, Score, Lives)
  Branch(EnemiesRemaining <= 0) -> True:
    GameActive = false
    Call SetStatus(HUDWidget, "You Win! Press R to Restart")

Custom Event HandleEnemyEscaped ->
  Lives = Lives - 1
  Call UpdateHUD(HUDWidget, Score, Lives)
  Branch(Lives <= 0) -> True:
    GameActive = false
    Call SetStatus(HUDWidget, "Game Over - Press R to Restart")

Enhanced Input Action Event (IA_Restart), Started ->
  Branch(!GameActive) -> True: Call ResetGame
```

The Round Ends - Then It Starts Fresh

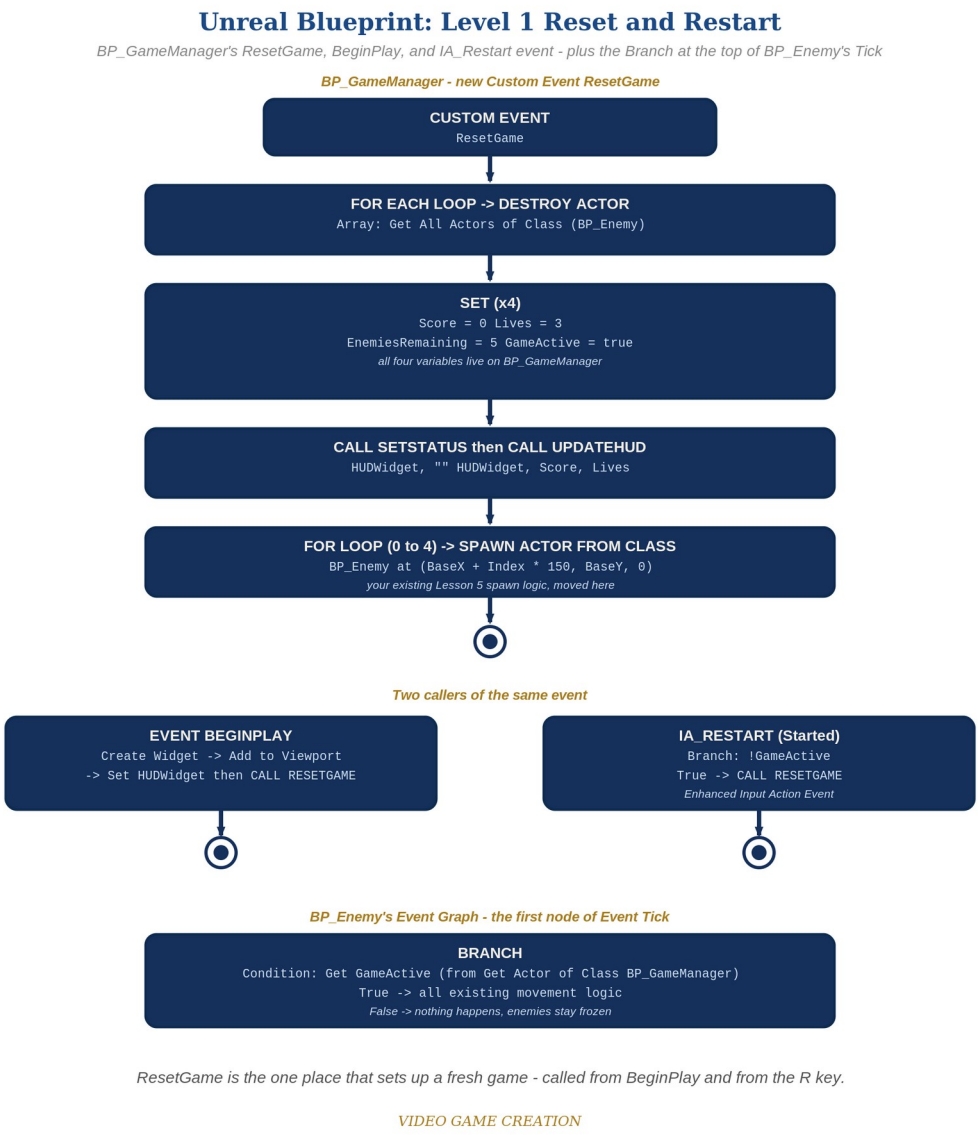
The same shape in every engine, whatever the exact code or blocks look like



start_game also runs once at launch - one setup routine, two doorways in.
Nothing about the setup logic was rewritten for restarting.

VIDEO GAME CREATION

When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.



BP_GameManager's ResetGame, BeginPlay and IA_Restart event, plus the Branch at the top of BP_Enemy's Tick - the node-for-node equivalent of the Level 1 code.

Why This Structure

- Reusing WBP_HUD (rather than falling back to a fresh Print String) keeps this lesson consistent with Lesson 9's actual UI investment - the win/lose/restart message now lives in the same persistent widget as score and lives, not a separate, older mechanism.
- Wrapping Event Tick's movement logic in Branch(GameActive) is the same pattern used in every scripted engine this week (Godot's early-return, Unity's early-return, GameMaker's early-exit) - Unreal's Blueprint equivalent is simply wrapping the relevant nodes in a Branch's True case instead of an early return, since Blueprint doesn't have a direct 'return' node mid-event the same way code does.

Level 2 / Level 3 Talking Points

- For students who did both Unity and Unreal: compare Unity's EnemySpawner.instance singleton against Unreal's Get All Actors of Class - two different mechanisms for the same underlying need ('find all the enemies').
- This is the most Blueprint-node-heavy lesson yet for Unreal - worth acknowledging directly with students working in this engine that today's ResetGame event graph will look busier than previous weeks, and that's expected given everything it's now responsible for.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: after a win and after a loss, R restores 0/3/5 with no leftover
enemies; R during play does nothing.
```

Level 3 - code (additions highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// Level 3 visual: WBP_HUD Designer view - StatusText Color and Opacity
set to a gold color; in the Graph, a Play Animation node on a Fade In
animation inside SetStatus.
WBP_HUD's Event Graph:
Custom Event SetStatus(StatusText: String) ->
  Set Text(StatusTextBlock, StatusText)
  Play Animation(FadeIn)
```

Common Student Mistakes

- Wiring the IA_Restart Enhanced Input Action Event's Started pin to call ResetGame directly, without the Branch(!GameActive) gate first - allows restarting mid-round.
- Forgetting to call SetStatus with an empty string inside ResetGame, leaving the old win/lose message visible after a restart even though gameplay has resumed.
- Leaving old enemy-spawning logic directly in BeginPlay instead of fully moving it into ResetGame - results in enemies being spawned twice at the very first game start.